CORS Vulnerability with Basic Origin Reflection

Lab Overview

- **Platform:** PortSwigger Web Security Academy
- **Vulnerability:** Cross-Origin Resource Sharing (CORS)
- **Difficulty:** Practitioner-level lab
- **Category:** Web Application Security

1. Description

Cross-Origin Resource Sharing (CORS) is a browser security mechanism that controls whether a web application can access resources from another origin.

This vulnerability occurs when a server reflects an arbitrary `Origin` header in the `Access-Control-Allow-Origin` response header without properly validating the requesting origin. If credentials are also permitted, a malicious website may be able to read sensitive information belonging to an authenticated user.

2. Vulnerability Identification

During testing, I inspected the application's HTTP requests and responses using Burp Suite. I modified the `Origin` header to determine whether the server trusted an unapproved external origin.

Example request:

```
GET /accountDetails HTTP/1.1
Host: target.example
Origin: https://attacker.example
Cookie: session=USER_SESSION
```

Example vulnerable response:

```
HTTP/1.1 200 OK
Access-Control-Allow-Origin: https://attacker.example
Access-Control-Allow-Credentials: true
Content-Type: application/json
```

The server reflected the supplied origin and allowed credentialed cross-origin access, indicating insufficient origin validation.

Note: The domain, endpoint, and response above are illustrative examples. Actual lab evidence should match the tested application.

3. Testing Methodology

1. Opened the lab and identified a potentially sensitive application endpoint.
2. Captured the relevant request using Burp Suite Proxy.
3. Sent the request to Repeater.
4. Added an external value to the `Origin` header.
5. Examined the response for `Access-Control-Allow-Origin` and `Access-Control-Allow-Credentials`.
6. Evaluated whether the browser could expose the response to an untrusted origin.

4. Security Impact

If exploitable in a real application, this issue could allow an attacker-controlled website to read sensitive cross-origin responses, such as account information or personal data, when a victim is authenticated and the browser sends the required credentials.

The actual impact depends on the exposed endpoint, browser behavior, authentication mechanism, and the application's CORS configuration.

5. Remediation

- Allow only explicitly trusted origins.
- Never reflect arbitrary `Origin` values without validation.
- Enable credentialed CORS access only when necessary.
- Avoid exposing sensitive data through cross-origin endpoints.
- Use `Vary: Origin` where responses vary according to the request origin.
- Test CORS policies against unexpected origins.

6. Key Takeaways

This lab demonstrated how improper CORS origin validation can expose sensitive resources to untrusted websites. Reviewing response headers and testing origin handling with Burp Suite are important steps in identifying CORS misconfigurations.

Tools Used: Burp Suite, browser developer tools, PortSwigger Web Security Academy.